

UML Diagrams

NexusNotes

Centralized Academic Resource Platform

Project Team: Code Club

Ramasani Adi Pranav – 24CSB0A57

Syed Mohammad Sami – 24CSB0A77

Janapala Harsha Vardhan Reddy – 24CSB0A28

Version 1.0

March 20, 2026

Contents

1	Introduction	2
2	Use Case Diagram	2
2.1	Explanation	4
3	Sequence Diagram: Upload Academic Resource (UC-01)	4
3.1	Explanation	4
4	Collaboration Diagram: Search and Download Resource (UC-02)	5
4.1	Explanation	5
5	Activity Diagram: Content Moderation (UC-06)	6
5.1	Explanation	8
6	Software Architecture Overview	8
7	Logical View: Class Diagram	8
7.1	Explanation	9
8	Logical View: Statechart Diagram	10
8.1	Explanation	12
9	Component View: Component Diagram	12
9.1	Explanation	12
10	Deployment View: Deployment Diagram	13
10.1	Explanation	13

1 Introduction

This document provides a comprehensive overview of the Unified Modeling Language (UML) diagrams for the Nexus Notes platform. Nexus Notes is a centralized academic knowledge platform that integrates students, faculty, and academic resources into a unified digital ecosystem. The following sections detail the Use Case, Sequence, Collaboration, and Activity diagrams that map out the system's architecture and user interactions.

2 Use Case Diagram

The Use Case Diagram provides a high-level overview of the interactions between the system's actors and its core functionalities.

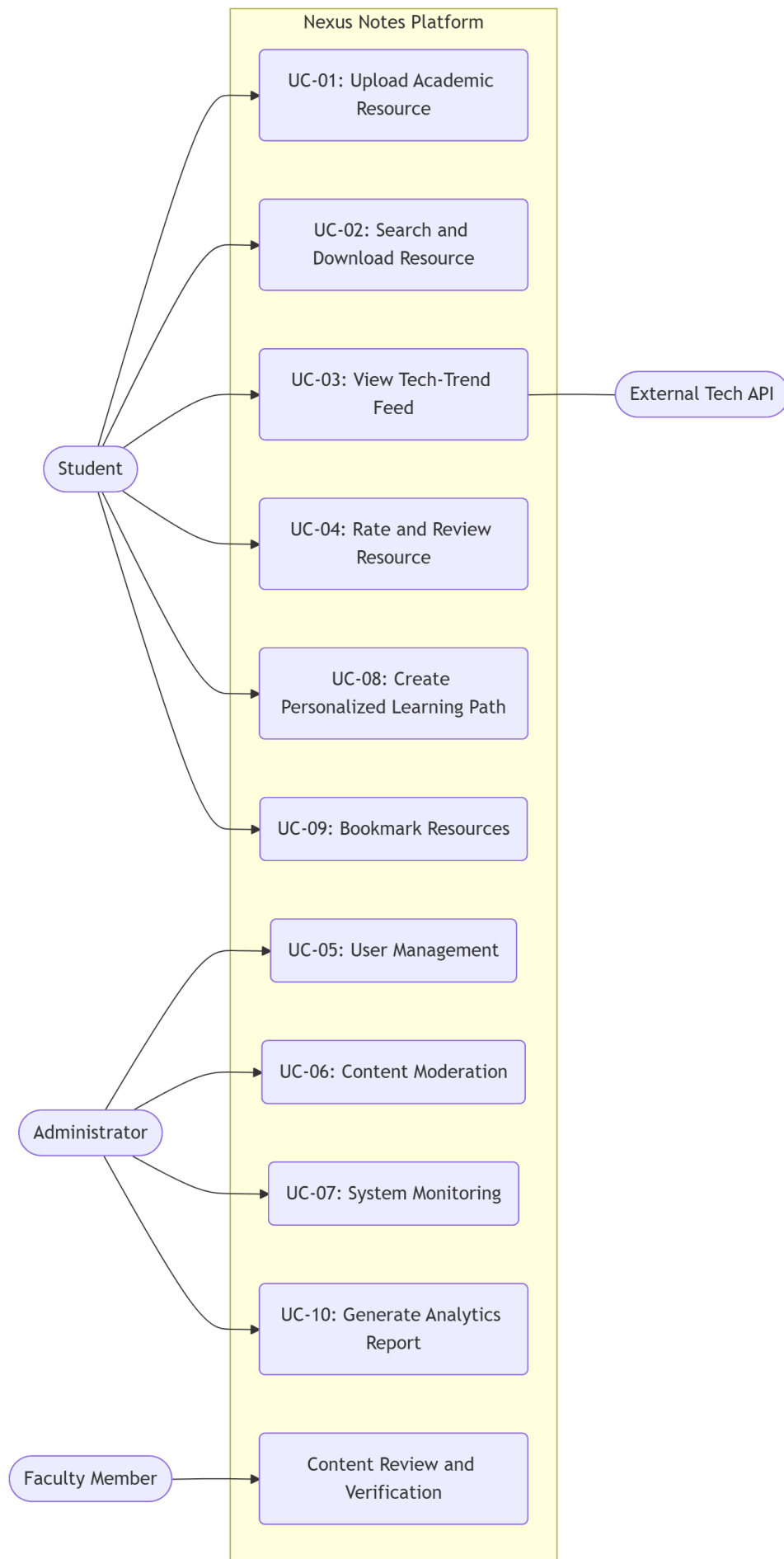


Figure 1: Use Case Diagram³ for Nexus Notes Platform

2.1 Explanation

Based on the system requirements, the primary actors in the Nexus Notes platform include the Student, Administrator, Faculty Member, and an External Tech API.

- **Student:** Interacts with features such as uploading academic resources (UC-01), searching and downloading resources (UC-02), viewing the tech-trend feed (UC-03), and rating resources (UC-04).
- **Administrator:** Responsible for user management (UC-05), content moderation (UC-06), and system monitoring (UC-07).
- **Faculty Member:** Acts as a secondary user to review and verify academic content.
- **External Tech API:** A simple actor that the system interacts with to fetch real-time technological updates for the Tech-Trend Feed.

3 Sequence Diagram: Upload Academic Resource (UC-01)

The Sequence Diagram details the chronological step-by-step process of uploading an academic resource.

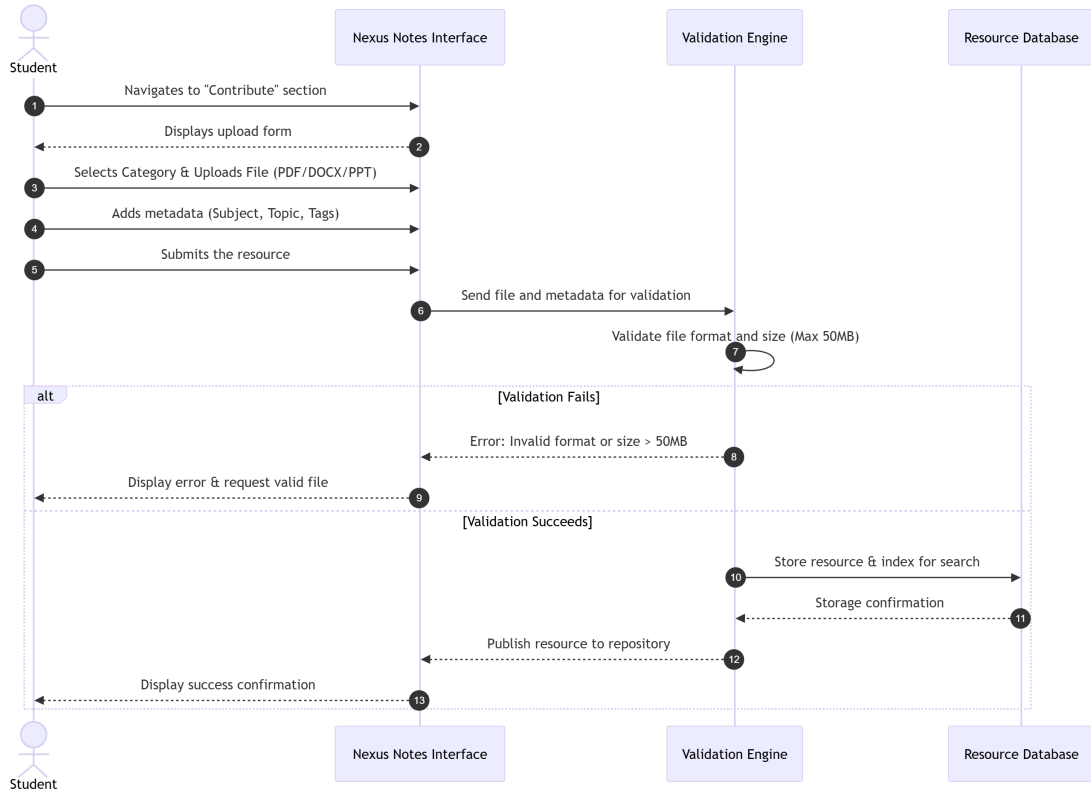


Figure 2: Sequence Diagram for UC-01: Upload Academic Resource

3.1 Explanation

This diagram illustrates the normal workflow and possible exceptions for UC-01.

- The student navigates to the "Contribute" section and the UI displays the upload form.
- The student selects the category, uploads a file (PDF, DOCX, PPT), adds metadata, and submits it.
- The system's validation engine checks if the file format is valid and whether the file size is under the 50MB limit.
- If validation fails (for example, due to an unsupported file type), an error message is displayed requesting a valid format.
- If validation succeeds, the resource is stored and indexed in the database, and a success confirmation is displayed to the student.

4 Collaboration Diagram: Search and Download Resource (UC-02)

The Collaboration Diagram (represented as a numbered communication flow) shows the structural message passing between objects during a resource search and download.

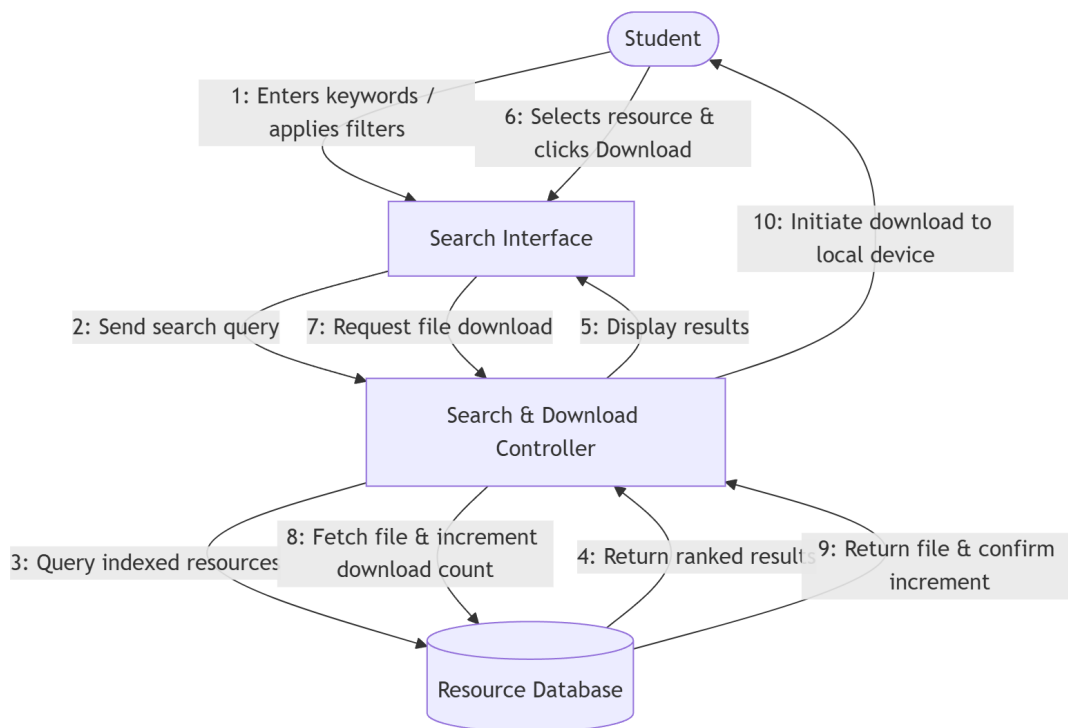


Figure 3: Collaboration Diagram for UC-02: Search and Download Resource

4.1 Explanation

This diagram highlights how system components interact when a student searches for study materials using keywords or filters.

- The flow begins with the student entering keywords or applying filters (such as Subject or Module) through the Search Interface.
- The query is passed to the Controller, which retrieves relevant results from the indexed Resource Database.
- The database returns ranked results based on relevance and user ratings, which are then displayed to the student.
- When the student selects a resource and clicks "Download", the system fetches the file, increments the download count in the database, and initiates the download to the student's local device.

5 Activity Diagram: Content Moderation (UC-06)

The Activity Diagram illustrates the control flow involved in the administrative task of moderating content.

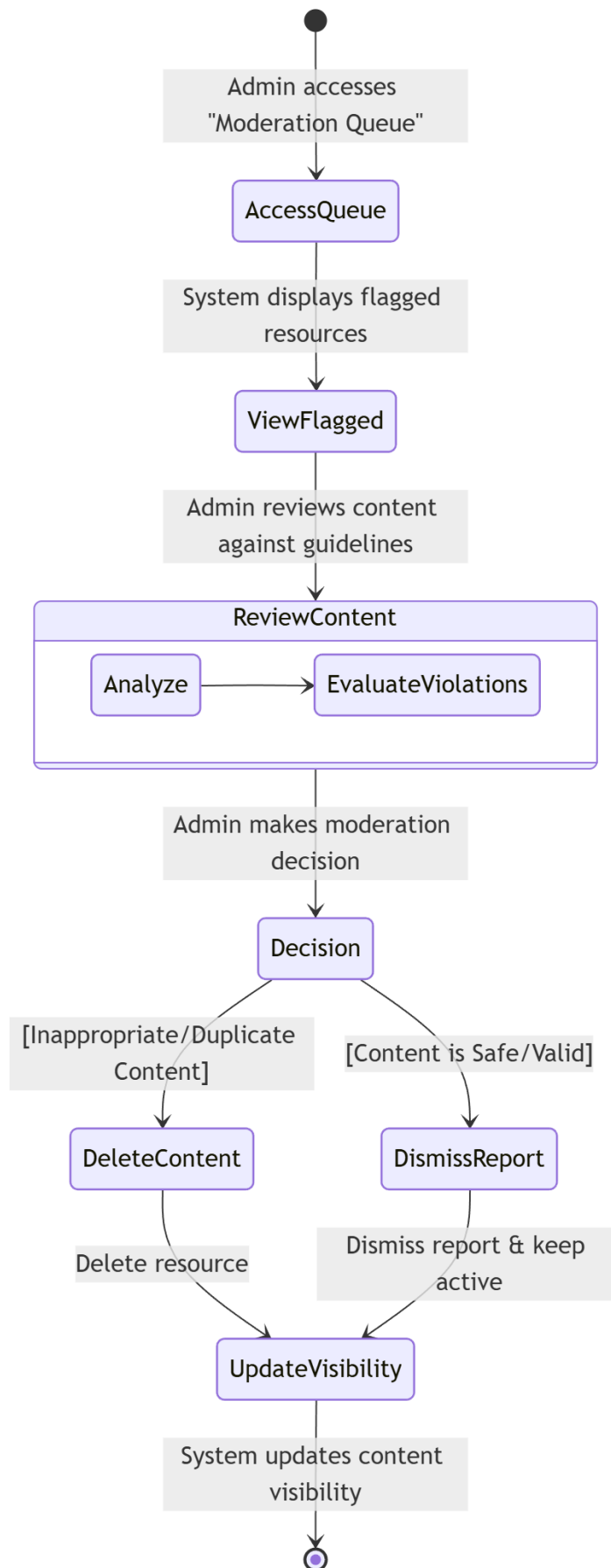


Figure 4: Activity Diagram for UC-06: Content Moderation

5.1 Explanation

This diagram outlines the decision-making process for administrators when managing flagged content.

- The Administrator accesses the Moderation Queue where the system displays flagged resources.
- The administrator reviews the content according to the platform's community guidelines.
- A moderation decision is made. If the content is inappropriate or duplicated, the administrator deletes the resource. If the report is invalid, the administrator dismisses the report and keeps the content active.
- Finally, the system updates the visibility status of the content based on the administrator's action.

6 Software Architecture Overview

This document details the software architecture for the Nexus Notes platform. It outlines the Logical View, Component View, and Deployment View using Unified Modeling Language (UML) diagrams. The architecture is built around a modern MERN stack (MongoDB, Express.js, React, Node.js) to support a robust, student-centric academic resource ecosystem.

7 Logical View: Class Diagram

The Class Diagram maps the object-oriented structure of the Nexus Notes system, defining the core entities, their attributes, methods, and relationships.

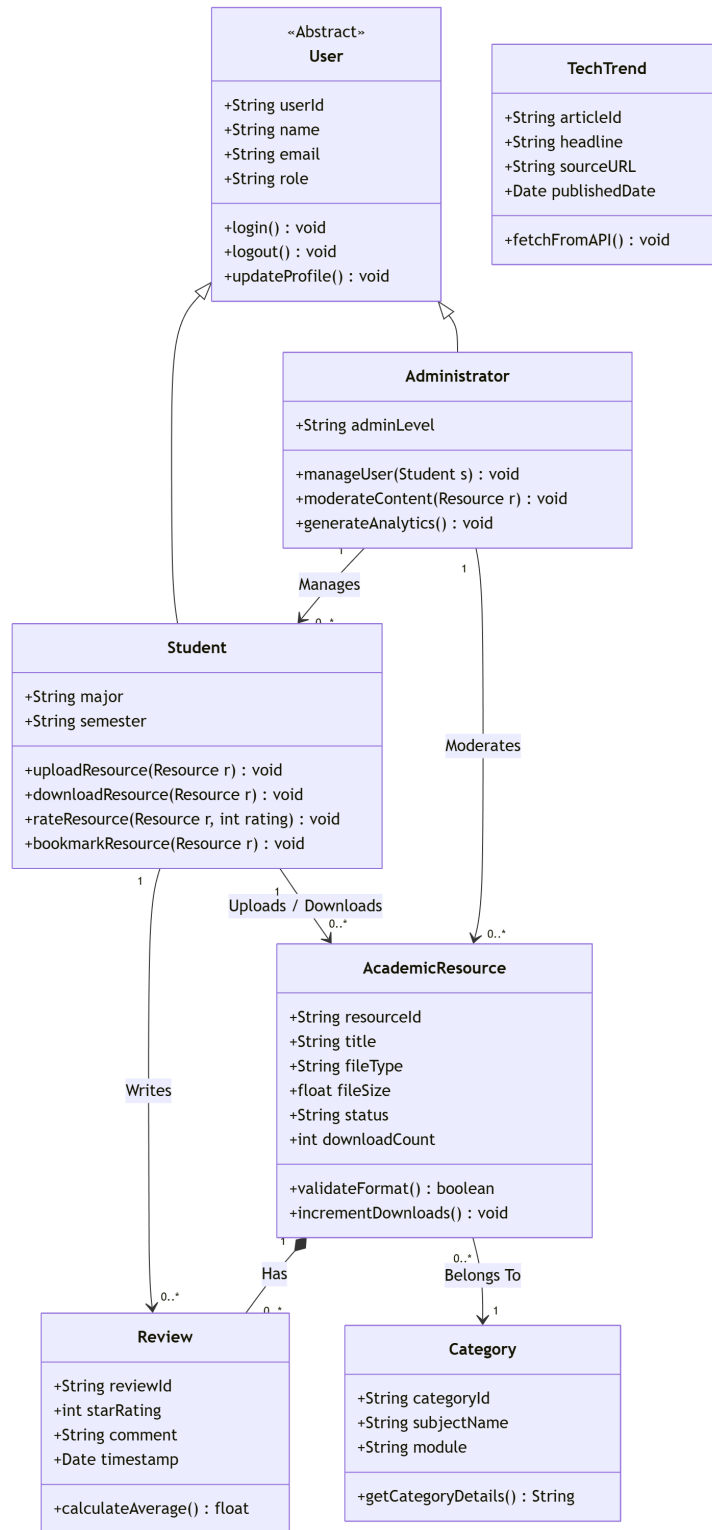


Figure 5: Logical View: Class Diagram

7.1 Explanation

The system’s data structure revolves around the primary stakeholders—the students—and the resources they interact with.

- **User Hierarchy:** The abstract **User** class provides base attributes (ID, name,

email) and authentication methods. The **Student** class inherits from this base, adding specific behaviors like uploading, downloading, and bookmarking resources. The **Administrator** class also inherits from **User** and handles moderation and user management.

- **AcademicResource & Category:** The **AcademicResource** class encapsulates file metadata (size, format, download count) and lifecycle methods. It maintains a many-to-one relationship with the **Category** class, ensuring all materials are strictly organized by subject and module.
- **Review System:** The **Review** class acts as a transactional entity linked to **AcademicResource**, storing star ratings and comments left by students to drive the platform's intelligent ranking system.
- **TechTrend:** A standalone entity that models the data fetched from the external APIs to keep students informed on emerging technologies.

8 Logical View: Statechart Diagram

The Statechart Diagram illustrates the lifecycle and varying states of an **AcademicResource** within the Nexus Notes platform.

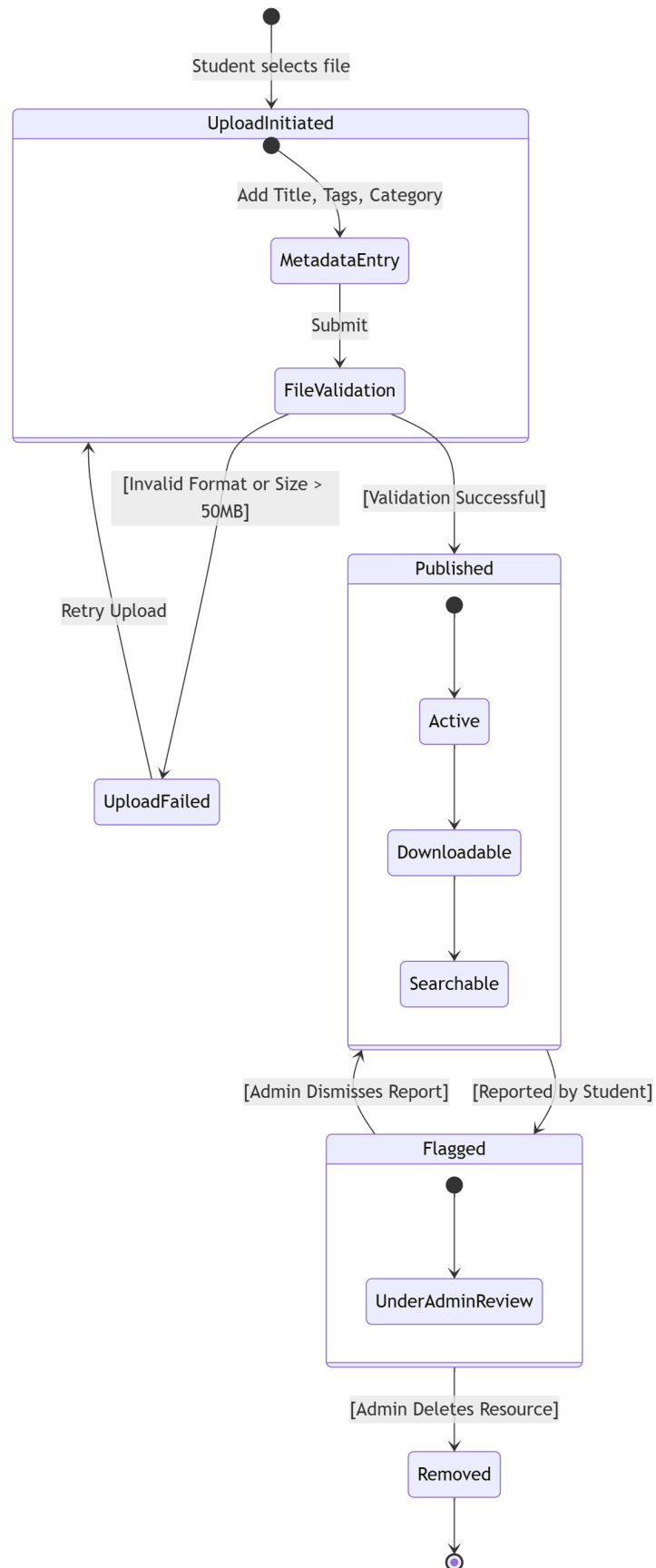


Figure 6: Logical View: Statechart Diagram

8.1 Explanation

This diagram tracks the dynamic behavior of an academic file from the moment a student initiates an upload.

- **Upload & Validation:** The process begins in the `UploadInitiated` state. Once the student inputs the required metadata, the system transitions to `FileValidation`. If the file exceeds 50MB or uses an unsupported format, it enters the `UploadFailed` state, prompting a retry.
- **Active State:** Upon successful validation, the resource transitions to `Published`. Here, it simultaneously becomes `Downloadable` and `Searchable` for the student body.
- **Moderation Flow:** If a student reports the content, the resource shifts to the `Flagged` state, triggering an `UnderAdminReview` sub-state. Depending on the administrator's evaluation, it is either returned to the `Published` state or permanently transitioned to `Removed`.

9 Component View: Component Diagram

The Component Diagram details the modular structure of the Nexus Notes software application, mapped specifically to the MERN stack environment.

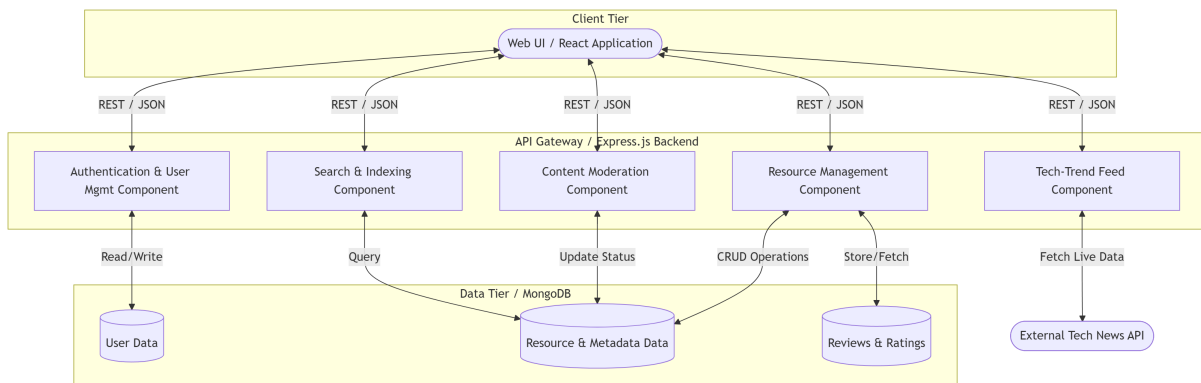


Figure 7: Component View: Component Diagram

9.1 Explanation

The system is divided into three highly decoupled tiers to ensure maintainability and scalability.

- **Client Tier:** The Web UI is a Single Page Application (SPA) built with React. It handles all user interactions and dynamic rendering of dashboards.
- **Application Tier (API Gateway):** The Node.js and Express.js backend serves as the bridge. It is composed of independent functional modules: Authentication, Resource Management, Search & Indexing, Content Moderation, and the Tech-Trend Feed component (which communicates securely with external APIs).

- **Data Tier:** The persistent storage is handled by MongoDB. Data is logically separated into **User Data**, **Resource & Metadata Data**, and **Reviews & Ratings** databases to optimize query times during complex searches.

10 Deployment View: Deployment Diagram

The Deployment Diagram visualizes the physical execution architecture and how software artifacts are distributed across hardware nodes.

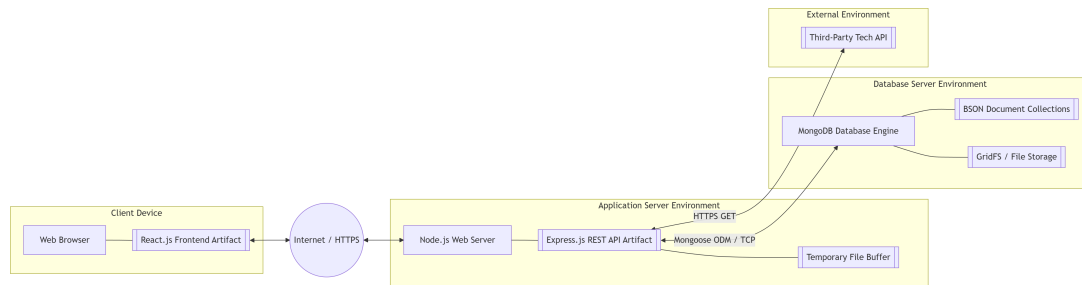


Figure 8: Deployment View: Deployment Diagram

10.1 Explanation

The deployment topology outlines how students access the system and how the servers process those requests.

- **Client Device:** Represents the student's hardware (laptop, tablet, or smartphone). The execution environment is a standard Web Browser running the compiled React.js frontend artifact.
- **Application Server Environment:** A cloud-hosted Node.js web server running the Express.js REST API. This node handles all business logic, routing, and temporary file buffering during uploads.
- **Database Server Environment:** A dedicated MongoDB server cluster. It utilizes BSON document collections for highly flexible metadata storage and GridFS for chunking and storing the actual academic resource files (PDFs, PPTs) natively within the database structure.
- **External Environment:** Represents the third-party Tech API servers, accessed asynchronously over HTTPS by the application server to populate the student dashboard.